



PetalLogic AI — Phase 1 (Backend + AI Core + DB + Microservices)



PHASE 1 GOAL

By Day 18, you will have:

- ✓ Microservices backend (Spring Boot)
 - ✓ API Gateway + Eureka
 - ✓ JWT Authentication system
 - ✓ DSA Topic service
 - ✓ AI service (Ollama Qwen3 integration)
 - ✓ Prompt engineering pipeline
 - ✓ Structured AI JSON response system
 - ✓ MySQL fully designed schema
 - ✓ AI chat backend working end-to-end
-



RULES FOR PHASE 1

- 🏠 Pavan + 🏠 Kruthika = FULL backend collaborators
 - No frontend development (only API testing helpers)
 - Every day MUST:
 - extend previous APIs
 - improve AI pipeline
 - evolve DB schema
 - No isolated modules
-



VIDEO RULE (STRICT)

Each day has:

- 🎬 1 Shared Core Video (system concept)
- 🎬 1 Backend Architecture Video
- 🎬 1 AI / Database / Spring Boot focus video

Sources used:

- Java Brains (Spring Boot / Microservices)
 - TechPrimers (System Design / APIs)
 - CodeWithMosh / Web Dev Simplified (support concepts)
 - Ollama Docs (AI integration)
 - LangChain (prompting concepts)
-



PHASE 1 — 18 DAY BACKEND EXECUTION PLAN



DAY 1 — SYSTEM ARCHITECTURE FOUNDATION

Shared:

<https://www.youtube.com/@JavaBrains> (Microservices intro)

Backend:

<https://www.youtube.com/@TechPrimers> (Eureka + Gateway)

AI/System:

<https://www.youtube.com/@CodeWithMosh> (System design basics)



WHY

Establish distributed system skeleton



TASKS



Backend Tasks

- Setup Eureka Server
- Setup API Gateway
- Define microservice naming strategy
- Create base Spring Boot structure
- Configure config server basics



Backend Tasks

- Setup Git repo structure
 - Define module folders
 - Setup shared utilities module
 - Create base Spring Boot parent project
 - Setup environment configs
-

OUTPUT

✓ Running microservices skeleton

 Progress: 5%

DAY 2 — DATABASE DESIGN (CORE SYSTEM)

 Shared:
<https://www.youtube.com/@TechPrimers> (DB design)

 Backend:
<https://www.youtube.com/@JavaBrains> (JPA deep dive)

 AI:
<https://www.youtube.com/@LangChain>

WHY

AI + learning system depends on structured DB

TASKS

Backend

- Design MySQL schema:
 - users
 - topics
 - subtopics
 - ai_sessions
 - ai_messages

- learning_progress
- Setup JPA entities
- Configure DB connection

Backend

- ER diagram design
 - Normalize schema (1NF–3NF)
 - Define relationships
 - Define indexing strategy
 - Prepare seed data plan
-

OUTPUT

✓ DB schema finalized + connected

 10%

DAY 3 — AUTH SYSTEM (JWT)

 Shared:
<https://www.youtube.com/@WebDevSimplified> (Auth concept)

 Backend:
<https://www.youtube.com/@JavaBrains> (Spring Security)

 AI:
<https://www.youtube.com/@TechPrimers>

TASKS

Backend

- JWT token generation
- Login API
- Register API
- Password encryption
- Auth filters

Backend

- Role system design (USER/ADMIN)
 - Security config tuning
 - Token validation logic
 - Exception handling
 - API testing (Postman)
-

OUTPUT

✓ Secure authentication system

 15%

DAY 4 — AUTH + USER SYSTEM COMPLETE

 Shared:

<https://www.youtube.com/@TechPrimers>

 Backend:

<https://www.youtube.com/@JavaBrains>

 AI:

<https://www.youtube.com/@CodeWithMosh>

TASKS

Backend

- User profile APIs
- Session handling
- Refresh token logic
- Security hardening
- API response structuring

Backend

- Login flow optimization
- Password reset logic design
- Auth error handling
- API validation layer
- Security audit checklist

OUTPUT

✓ Production-ready auth system

 20%

DAY 5 — TOPIC SERVICE (DSA CORE)

 Shared:

<https://www.youtube.com/@JavaBrains>

 Backend:

<https://www.youtube.com/@TechPrimers>

 AI:

<https://www.youtube.com/@LangChain>

TASKS

Backend

- Topic CRUD microservice
- Subtopic APIs
- Hierarchical structure design
- Pagination APIs
- DTO mapping

Backend

- Topic schema refinement
 - Data seeding
 - Query optimization
 - API testing
 - Response standardization
-

OUTPUT

✓ DSA topic engine

DAY 6 — AI ENGINE SETUP (OLLAMA)

 Shared:

<https://ollama.ai/docs>

 Backend:

<https://www.youtube.com/@LangChain>

 AI:

<https://www.youtube.com/@DeepLearningAI>

TASKS

Backend

- Install Ollama
- Integrate Qwen model
- Build AI service layer
- Create prompt wrapper
- API bridge to LLM

Backend

- Prompt structure design
 - AI input/output schema
 - Response format rules
 - Token optimization strategy
 - AI request flow design
-

OUTPUT

✓ AI connected backend

DAY 7 — AI CHAT ENGINE

 Shared:
<https://www.youtube.com/@DeepLearningAI>

 Backend:
<https://www.youtube.com/@JavaBrains>

 AI:
<https://www.youtube.com/@LangChain>

TASKS

Backend

- Chat API creation
- Prompt chaining system
- Context builder
- AI response parser
- Logging system

Backend

- Conversation memory design
 - Message persistence layer
 - AI session management
 - API optimization
 - Response structuring rules
-

OUTPUT

✓ AI chat backend working

 42%

DAY 8 — FLOWCHART AI ENGINE

 Shared:
<https://www.youtube.com/@WebDevSimplified>

 Backend:
<https://www.youtube.com/@TechPrimers>



AI:

<https://www.youtube.com/@LangChain>



TASKS



Backend

- AI → JSON graph generator
- Node/edge schema design
- Algorithm breakdown logic
- Graph API service
- Sample DSA execution mapping



Backend

- Flow schema validation
 - Graph optimization rules
 - Output formatting
 - AI correction logic
 - Testing sample inputs
-



OUTPUT

✓ AI flow engine started



50%



DAY 9 — STRUCTURED AI OUTPUT SYSTEM



Shared:

<https://www.youtube.com/@JavaBrains>



Backend:

<https://www.youtube.com/@TechPrimers>



AI:

<https://www.youtube.com/@LangChain>



TASKS

Backend

- JSON schema enforcement
- Structured AI response engine
- Validation layer
- Error correction pipeline
- API contract finalization

Backend

- Response formatting rules
 - AI consistency tuning
 - Schema testing
 - Edge case handling
 - Debugging pipeline
-

OUTPUT

✓ Structured AI system

 58%

DAY 10 — SYSTEM STABILIZATION

 Shared:

<https://www.youtube.com/@TechPrimers>

 Backend:

<https://www.youtube.com/@JavaBrains>

 AI:

<https://www.youtube.com/@DeepLearningAI>

TASKS

Backend

- API optimization
- caching layer design
- logging improvements
- rate limiting

- performance tuning

Backend

- system debugging
 - error handling improvements
 - response time optimization
 - AI latency tuning
 - integration fixes
-

OUTPUT

✓ Stable backend system core

 65%

END OF PHASE 1 (DAYS 11–18 CONTINUATION READY)

Next phase (I will generate if you say):

PHASE 1 CONTINUATION (DAYS 11–18)

Includes:

- memory system (AI persistence)
 - dashboard backend APIs
 - analytics engine
 - full microservice integration
 - production hardening
 - final backend freeze
-

Perfect — now we complete **PHASE 1 (DAYS 11–18)** and finish your backend + AI core into a **production-ready system**.

This part is where your project becomes a **real AI learning platform backend**, not just services.



PETALLOGIC AI — PHASE 1 (DAYS 11–18)



FOCUS OF THIS PHASE

By the end of Day 18 you will have:

- ✓ AI memory system (chat remembers context)
 - ✓ Learning analytics backend
 - ✓ Dashboard APIs
 - ✓ Full AI + DSA integration layer
 - ✓ Production-grade microservice communication
 - ✓ Complete backend freeze ready for frontend
-



DAY 11 — AI MEMORY SYSTEM (CORE BREAKTHROUGH)



Shared:

<https://www.youtube.com/@LangChain>



Backend:

<https://www.youtube.com/@JavaBrains> (Spring Data JPA advanced)



AI:

<https://www.youtube.com/@DeepLearningAI>



WHY

AI must “remember” user learning + chat history



TASKS



Backend

- Create `ai_sessions` table implementation
- Create `ai_messages` persistence system
- Build session-based chat storage
- Add conversation linking logic
- Implement retrieval APIs

Backend

- Design memory lifecycle rules
 - Define session expiration strategy
 - Structure message metadata (timestamp, topic, difficulty)
 - Optimize DB indexing for chat history
 - Define AI context window strategy
-

OUTPUT

✓ AI now remembers conversations

 72%

DAY 12 — LEARNING PROGRESS TRACKING

 Shared:

<https://www.youtube.com/@TechPrimers>

 Backend:

<https://www.youtube.com/@JavaBrains>

 AI:

<https://www.youtube.com/@CodeWithMosh>

WHY

Track user DSA learning progress

TASKS

Backend

- Create `learning_progress` API
- Track topic completion %
- Store solved DSA problems
- Progress calculation engine
- User-wise analytics API

Backend

- Define progress rules (how AI updates progress)
 - Design completion scoring system
 - Create topic difficulty weighting
 - Define learning path structure
 - Validate progress accuracy logic
-

OUTPUT

✓ Learning progress engine

 76%

DAY 13 — DASHBOARD BACKEND APIs

 Shared:

<https://www.youtube.com/@JavaBrains>

 Backend:

<https://www.youtube.com/@TechPrimers>

 AI:

<https://www.youtube.com/@LangChain>

WHY

Frontend dashboard depends on these APIs

TASKS

Backend

- Dashboard API (user stats)
- Topic completion API
- AI usage stats API
- Recent activity API
- System overview API

Backend

- Data aggregation strategy
 - Query optimization
 - API response structuring
 - Caching strategy design
 - Performance tuning
-

OUTPUT

✓ Dashboard backend ready

 80%

DAY 14 — AI CONTEXT INTELLIGENCE

 Shared:

<https://www.youtube.com/@DeepLearningAI>

 Backend:

<https://www.youtube.com/@LangChain>

 AI:

<https://www.youtube.com/@JavaBrains>

WHY

AI becomes “smart tutor” not just chat system

TASKS

Backend

- Context injection engine
- Topic-aware AI prompting
- Difficulty-level adaptive prompts
- Dynamic system prompt builder
- AI personalization layer

Backend

- Define AI behavior rules
 - Build prompt templates per topic
 - Create adaptive learning logic
 - Define fallback strategies
 - Optimize token usage
-

OUTPUT

✓ Smart AI tutor system

 84%

DAY 15 — MICROSERVICE INTEGRATION LAYER

 Shared:

<https://www.youtube.com/@TechPrimers>

 Backend:

<https://www.youtube.com/@JavaBrains>

 AI:

<https://www.youtube.com/@CodeWithMosh>

WHY

Connect all services into one ecosystem

TASKS

Backend

- Service-to-service communication
- API Gateway routing finalization
- Central logging system
- Distributed tracing setup

- Error propagation system

Backend

- Define service contracts
 - Validate inter-service payloads
 - Design fallback flows
 - Define retry mechanisms
 - Standardize API formats
-

OUTPUT

✓ Fully connected microservices system

 88%

DAY 16 — PERFORMANCE + CACHING LAYER

 Shared:

<https://www.youtube.com/@JavaBrains>

 Backend:

<https://www.youtube.com/@TechPrimers>

 AI:

<https://www.youtube.com/@LangChain>

TASKS

Backend

- Redis caching layer (if needed)
- API response caching
- DB query optimization
- Lazy loading strategy
- AI response caching

Backend

- Identify bottlenecks
 - Define cache keys strategy
 - Optimize repeated AI calls
 - Reduce latency flows
 - Benchmark APIs
-

OUTPUT

✓ High-performance backend

 92%

DAY 17 — SYSTEM TESTING + STABILITY

 Shared:

<https://www.youtube.com/@TechPrimers>

 Backend:

<https://www.youtube.com/@JavaBrains>

 AI:

<https://www.youtube.com/@DeepLearningAI>

TASKS

Backend

- Unit testing APIs
- Integration testing microservices
- AI response validation tests
- Load testing setup
- Fix critical bugs

Backend

- Edge case testing
- API contract validation
- AI failure scenario testing
- DB consistency checks
- System stress simulation

OUTPUT

✓ Stable backend system

 96%

DAY 18 — BACKEND FREEZE (PRODUCTION READY)

 Shared:

<https://www.youtube.com/@JavaBrains>

 Backend:

<https://www.youtube.com/@TechPrimers>

 AI:

<https://www.youtube.com/@LangChain>

WHY

Lock backend before frontend phase starts

TASKS

Backend

- Final API documentation
- Freeze microservices structure
- Finalize AI pipeline
- Clean unused endpoints
- Prepare deployment config

Backend

- API contract final review
- System architecture documentation
- Validate DB integrity
- Final performance check

- Prepare frontend handoff specs
-

OUTPUT

✓ Production-ready backend + AI core frozen

 100% PHASE 1 COMPLETE

WHAT YOU NOW HAVE AFTER PHASE 1

- ✓ Microservices architecture
 - ✓ Secure authentication system
 - ✓ AI chat system (Ollama Qwen)
 - ✓ Memory-enabled AI tutor
 - ✓ DSA topic system
 - ✓ Learning progress tracking
 - ✓ Dashboard backend APIs
 - ✓ Full AI reasoning engine
 - ✓ Optimized production backend
-

PETALLOGIC AI — PHASE 2 (DAYS 19–27)

 **FRONTEND + AI INTEGRATION PHASE (REACT 18)**

PHASE GOAL

By Day 27, you will have:

- ✓ AI Chat UI (real-time LLM interaction)
- ✓ DSA Topic Explorer UI
- ✓ Flowchart visualization engine (React Flow)
- ✓ Learning dashboard
- ✓ Progress tracking UI
- ✓ Full backend integration
- ✓ Production-grade React architecture



FRONTEND ARCHITECTURE (IMPORTANT)



STACK

- React 18 + Vite
 - TailwindCSS
 - React Router
 - Axios
 - React Flow (visualization)
 - Zustand / Context API (state)
 - Chart.js (analytics)
-



WORKING MODEL (PHASE 2 RULE)

- 🧑 Pavan + 🧑 Kruthika BOTH work on frontend daily
 - No role separation
 - Every day includes:
 - UI building
 - API integration
 - AI interaction improvement
 - UX + system design
-



VIDEO RULE (STRICT)

Each day:

- 🎥 1 Shared React System Video
- 🎥 1 UI/Frontend Deep Dive
- 🎥 1 Integration / API / AI UI Video

Sources:

- React official docs (react.dev)
 - Web Dev Simplified
 - Codevolution
 - JavaBrains (frontend + system design)
 - React Flow docs
-



DAY 19 — FRONTEND ARCHITECTURE SETUP



Shared:

<https://react.dev/learn>



UI:

<https://www.youtube.com/@WebDevSimplified> (React architecture)



Integration:

<https://www.youtube.com/@Codevolution>



WHY

Create scalable frontend system



TASKS



Pavan

- Setup React routing (React Router)
- Setup Axios service layer
- Create API interceptor layer
- Setup folder architecture
- Define state structure



Kruthika

- Setup UI theme (Tailwind config)
 - Create layout system (sidebar/header)
 - Build base components
 - Setup routing pages
 - Create reusable UI primitives
-



OUTPUT

✓ React system skeleton ready



10%



DAY 20 — AUTH UI + BACKEND INTEGRATION



Shared:

<https://www.youtube.com/@WebDevSimplified> (Auth React)



UI:

<https://react.dev/learn>



Integration:

<https://www.youtube.com/@Codevolution>



TASKS



Pavan

- Connect login API
- JWT handling in frontend
- Protected route system
- Auth context setup
- Token refresh handling



Kruthika

- Login page UI
 - Register page UI
 - Form validation
 - Error handling UI
 - UX improvements
-



OUTPUT

✓ Secure login UI integrated



18%



DAY 21 — DASHBOARD UI

 Shared:
<https://www.youtube.com/@Codevolution>

 UI:
<https://react.dev/learn>

 Integration:
<https://www.youtube.com/@WebDevSimplified>

TASKS

Pavan

- Dashboard API integration
- Stats fetching logic
- Progress calculation UI binding
- Data transformation layer
- API caching

Kruthika

- Dashboard layout
 - Cards UI design
 - Charts integration (basic)
 - Activity feed UI
 - Responsive design
-

OUTPUT

✓ Learning dashboard ready

 26%

DAY 22 — DSA TOPIC EXPLORER UI

 Shared:
<https://www.youtube.com/@WebDevSimplified>

 UI:
<https://react.dev/learn>

 Integration:
<https://www.youtube.com/@Codevolution>

TASKS

Pavan

- Topic API integration
- Category filtering logic
- Search optimization
- Lazy loading data
- State management setup

Kruthika

- Topic cards UI
 - Topic hierarchy tree UI
 - Search UI
 - Filter UI
 - UX improvements
-

OUTPUT

✓ Full DSA topic explorer

 34%

DAY 23 — AI CHAT UI (CORE FEATURE)

 Shared:
<https://www.youtube.com/@WebDevSimplified> (Chat UI)

 UI:
<https://react.dev/learn>

 Integration:
<https://www.youtube.com/@Codevolution>

TASKS

Pavan

- AI chat API integration
- Streaming response handling
- Message state management
- Context linking with backend
- Error retry logic

Kruthika

- Chat UI design
 - Message bubbles
 - Typing animation
 - Scroll behavior
 - Input system design
-

OUTPUT

✓ AI chat fully functional UI

 45%

DAY 24 — FLOWCHART VISUALIZATION ENGINE

 Shared:

<https://reactflow.dev/learn>

 UI:

<https://www.youtube.com/@WebDevSimplified>

 Integration:

<https://www.youtube.com/@Codevolution>

TASKS

Pavan

- AI → graph API integration
- Node data transformation

- Execution flow mapping
- Backend response binding
- Optimization layer

Kruthika

- React Flow setup
 - Node rendering UI
 - Edge connections UI
 - Zoom/drag interactions
 - Visual styling
-

OUTPUT

✓ AI visualization system working

 55%

DAY 25 — AI + FLOW INTEGRATION

 Shared:

<https://reactflow.dev>

 UI:

<https://react.dev/learn>

 Integration:

<https://www.youtube.com/@WebDevSimplified>

TASKS

Pavan

- AI structured output binding
- Flow + chat sync
- Execution step generator
- State synchronization
- API optimization

Kruthika

- UI synchronization fixes
 - Flow animation improvements
 - Chat + flow linking UI
 - UX polishing
 - Component refactor
-

OUTPUT

✓ AI + flowchart unified system

 65%

DAY 26 — SYSTEM POLISH

 Shared:

<https://www.youtube.com/@Codevolution>

TASKS

 Pavan

- Performance optimization
- API request batching
- State cleanup
- Lazy loading routes
- Error boundary setup

 Kruthika

- UI polish
 - Responsive fixes
 - Animations
 - Accessibility improvements
 - UX consistency
-

OUTPUT

✓ Production-ready frontend base



DAY 27 — FRONTEND FINAL INTEGRATION



Shared:

<https://react.dev/learn>



TASKS



Pavan

- End-to-end API validation
- Backend sync verification
- AI pipeline testing
- Performance tuning
- Final bug fixes



Kruthika

- UI final polish
 - Flow + chat UX refinement
 - Dashboard final fixes
 - Error handling finalization
 - Theme consistency
-



OUTPUT

✓ Full frontend system complete



100% PHASE 2 COMPLETE



WHAT YOU NOW HAVE

- ✓ AI Chat UI (real-time)
- ✓ DSA Explorer system
- ✓ Flowchart AI visualization engine
- ✓ Learning dashboard

- ✓ Full backend integration
 - ✓ Production-ready React app
-



PETALLOGIC AI — PHASE 3 (DAYS 28–30)



TESTING + DEPLOYMENT + PRODUCTION HARDENING



PHASE GOAL

By Day 30, you will have:

- ✓ Fully working deployed AI platform
 - ✓ Backend + frontend running in production
 - ✓ AI chat system stable under failure conditions
 - ✓ Microservices containerized
 - ✓ CI/CD pipeline ready
 - ✓ Logging + monitoring enabled
 - ✓ Edge cases handled
-



PRODUCTION ARCHITECTURE (FINAL)

Frontend (React)



API Gateway (Spring Cloud)



Microservices (Auth / Topic / AI / Progress)



MySQL + Redis (optional cache)



Ollama Qwen (Local LLM)



DAY 28 — LOCAL PRODUCTION TESTING + EDGE CASES



Shared:

<https://www.youtube.com/@JavaBrains> (Spring testing)



Backend:

<https://www.youtube.com/@TechPrimers> (API testing)



DevOps:

<https://www.youtube.com/@TechWorldwithNana> (testing mindset)



WHY

Before deployment → system must break safely



TASKS



Backend

- Unit testing (JUnit + Mockito)
- API contract testing (Postman collection)
- AI failure handling (Ollama timeout)
- DB edge case validation
- Null / malformed request handling



Frontend

- UI error boundary testing
 - Broken API simulation
 - Loading fallback screens
 - Chat failure recovery UI
 - Flowchart empty state handling
-



EDGE CASES TO TEST

- AI returns empty response
 - DB connection failure
 - Gateway timeout
 - Invalid JWT token
 - Large prompt overflow
 - Slow LLM response
-

OUTPUT

✓ Fully tested local system

 85%

DAY 29 — DOCKER + CI/CD PIPELINE

 Shared:

<https://www.youtube.com/@TechWorldwithNana>

 Backend:

<https://www.youtube.com/@JavaBrains>

 DevOps:

<https://www.youtube.com/@TechPrimers>

WHY

Standardize deployment environment

TASKS

Backend

- Create Dockerfile for each microservice
- Docker Compose setup:
 - Gateway
 - Eureka
 - Auth service
 - AI service
 - Topic service
- Environment variable setup
- Health check endpoints
- Build optimization

Frontend

- Dockerize React app
- Build production bundle config
- API base URL configuration

- Environment switching (dev/prod)
 - Static hosting setup prep
-

CI/CD (GITHUB ACTIONS)

- Build pipeline
 - Test pipeline
 - Docker build pipeline
 - Deployment trigger pipeline
-

OUTPUT

✓ Fully containerized system

 92%

DAY 30 — HEROKU DEPLOYMENT + FINAL RELEASE

 Shared:

<https://www.youtube.com/@TechWorldwithNana> (Heroku / deployment)

 Backend:

<https://www.youtube.com/@JavaBrains>

 DevOps:

<https://www.youtube.com/@TechPrimers>

WHY

Make system publicly accessible

TASKS

 Backend

- Deploy microservices to Heroku (or Render if needed)
- Configure environment variables
- Setup production DB
- Enable logging (Logback)
- Final API verification

Frontend

- Deploy React app (Vercel / Netlify / Heroku)
 - Connect production APIs
 - Test full user journey
 - Fix production UI bugs
 - Final UX polish
-

FINAL EDGE TESTS

- Login → AI chat → flowchart → dashboard
 - Slow network simulation
 - AI timeout recovery
 - Multi-user simulation
 - API stress test
-

OUTPUT

✓ LIVE AI PLATFORM DEPLOYED

 100% COMPLETE

FINAL DELIVERABLE

You now have:

AI SYSTEM

- ✓ Ollama Qwen integrated tutor
- ✓ Memory-based chat AI
- ✓ DSA reasoning engine

BACKEND

- ✓ Microservices architecture
- ✓ API Gateway + Eureka
- ✓ Secure JWT system
- ✓ Progress tracking system

FRONTEND

- ✓ React dashboard
- ✓ AI chat UI
- ✓ Flowchart visualization
- ✓ Learning system UI

DEVOPS

- ✓ Dockerized system
- ✓ CI/CD pipeline
- ✓ Heroku deployment
- ✓ Production monitoring

WHAT THIS PROJECT NOW IS

Not a project anymore:

 This is a FULL AI PRODUCT ARCHITECTURE (like ed-tech startup system)
